

Table des Matieres

1. Resume Executif	2
2. Vulnerabilites de Securite Critiques	2
2.1 Acces non authentifie aux endpoints sensibles	2
2.2 Donnees simulees/falsifiees dans les reponses API	3
2.3 Exposition des emails de comptes de demonstration	3
2.4 Recherche d'opérateur par correspondance floue	4
2.5 Enregistrement d'alertes sans limitation de debit	4
3. Vulnerabilites de Securite Elevees	4
3.1 Limitation de debit en memoire non persistante	4
3.2 Base de donnees SQLite en production	4
3.3 Absence de strategie de migration de base de donnees	5
3.4 Credentials hardcodes dans les fichiers de configuration	5
3.5 Absence de pagination sur les endpoints de liste	5
3.6 Absence de validation CSRF explicite	5
4. Problemes d'Architecture	6
4.1 Monolithisme frontend et absence de routing serveur	6
4.2 Absence d'infrastructure de test	6
4.3 Duplication massive du code de mapping	6
4.4 Casts de type repetes et non securises	7
4.5 Absence de health check fonctionnel	7
4.6 Absence de gestion des transactions	7
4.7 Schema Prisma : index manquants	7
4.8 Absence de logging structure externe	7
4.9 Compatibilite des dependances	8
5. Qualite de Code	8
5.1 Points positifs identifies	8
5.2 Problemes de duplication de code	9
5.3 Gestion des erreurs insuffisante	9
5.4 Absence de documentation technique	9
5.5 Problèmes de configuration Docker	9
6. Recommandations Prioritaires	10
7. Matrice Complete des Vulnerabilites	11
8. Conclusion	13

1. Resume Executif

Ce rapport presente les resultats de l'audit complet de securite et de qualite de code de la plateforme **ONIT-PNG** (Observatoire National Intelligent des Telecommunications), une application Next.js destinee a la supervision des telecommunications en Republique de Guinee, developpee pour l'ARPT. L'audit a ete conduit sur l'ensemble du codebase comprenant les routes API, le schema Prisma, la configuration Docker, le middleware de securite, le systeme RBAC, et les composants frontend.

L'audit a revele **34 vulnerabilites et problemes** repartis en quatre categories : securite critique, securite elevee, qualite architecturale, et qualite de code. Parmi les decouvertes les plus preoccupantes, on note l'absence de verification d'authentification sur plusieurs endpoints API sensibles (dashboard, cartographie, scoring), la presence de donnees simulees melangees aux donnees reelles dans les reponses API, une base de donnees SQLite inadaptree a la production, et des problemes de validation des entrees qui pourraient permettre des attaques par injection ou des assignments de masse.

La plateforme presente neanmoins des fondations solides avec un systeme RBAC bien structure, une journalisation d'audit exhaustive, une validation Zod systematique sur les routes critiques, un nettoyage HTML (stripHtml) contre les attaques XSS, et un middleware de securite avec en-tetes de protection et limitation de debit. Cependant, plusieurs de ces mecanismes presentent des lacunes d'implementation qui necessitent une correction prioritaire avant tout deploiement en production.

Categorie	Nombre	Priorite
Securite Critique	8	Correction immediate requise
Securite Elevee	6	Correction avant mise en production
Architecture	9	Refactorisation planifiee
Qualite de Code	11	Amelioration continue

Tableau 1 : Synthese des vulnerabilites identifiees

2. Vulnerabilites de Securite Critiques

2.1 Acces non authentifie aux endpoints sensibles

Plusieurs routes API critiques ne verifient pas l'authentification de l'utilisateur avant de renvoyer des donnees sensibles. L'endpoint **GET /api/dashboard** constitue le cas le plus grave : il retourne des KPIs nationaux, le classement des operateurs, les alertes actives, et les statistiques regionales sans aucune verification de session. Si l'utilisateur n'est pas authentifie, la route definit un role par default "PUBLIC" et continue l'execution normale, retournant potentiellement des donnees confidentielles.

De meme, les endpoints **GET /api/map** et **GET /api/scoring** suivent le meme pattern dangereux : attribution d'un role "PUBLIC" en cas d'absence de session et poursuite du traitement. Bien que le systeme RLS (Row-Level Security) tente de filtrer les donnees selon le scope du role, la politique par default retourne "public_only" lorsqu'aucune politique n'existe, ce qui peut inadvertamment exposer plus de donnees que prevu. L'endpoint de scoring expose notamment les sous-scores detailles de chaque

opérateur, leurs tendances historiques, et leurs recommandations stratégiques.

Severite	Endpoint	Impact
CRITIQUÉ	GET /api/dashboard	Données KPI, alertes, scores opérateurs accessibles sans authentification
CRITIQUÉ	GET /api/map	Données cartographiques, points de mesure, couverture régionale sans authentification
CRITIQUÉ	GET /api/scoring	Scores détaillés, recommandations stratégiques des opérateurs sans authentification

Tableau 2 : Endpoints sans vérification d'authentification

Recommandation : Ajouter une vérification de session obligatoire en début de chaque handler GET. Retourner une erreur 401 si aucun utilisateur authentifié n'est détecté. Pour les données publiques légitimes, créer des endpoints distincts avec un filtrage explicite du contenu autorisé.

2.2 Données simulées/falsifiées dans les réponses API

L'endpoint **GET /api/dashboard** contient des manipulations de données inquiétantes qui faussent les indicateurs présents aux décideurs. La ligne de code suivante est particulièrement grave : le compteur de zones blanches est artificiellement augmenté de 200 unités avant d'être renvoyé au frontend. Cette pratique est qualifiée de falsification de données et constitue un risque majeur pour la crédibilité de l'observatoire.

```
zonesBlanches: { value: zonesBlanches + 200, ... }
```

De plus, la taille des fichiers de rapports est générée aléatoirement avec `Math.random()`, rendant les informations de taille complètes fictives. L'endpoint `/api/qos` utilise des valeurs de réplique codées en dur lorsque les données réelles sont insuffisantes, sans aucune indication visuelle que les données présentées sont estimées et non mesurées. Les tendances d'évolution des opérateurs sont simulées avec des valeurs prédéfinies si les données historiques sont insuffisantes.

Recommandation : Supprimer toute manipulation de données dans les réponses API. Les valeurs par défaut doivent être clairement identifiées comme telles dans la réponse JSON (ex: champ "estimated": true). Ajouter des métadonnées sur la source et la fiabilité de chaque indicateur.

2.3 Exposition des emails de comptes de démonstration

Le composant **login-modal.tsx** contient un tableau hardcode de 10 comptes de démonstration avec leurs adresses email complètes, leurs rôles et leurs organisations. Ces emails sont directement visibles dans le code JavaScript côté client, accessibles à quiconque consulte le code source de la page. Cette exposition facilite les attaques par ingénierie sociale et les tentatives d'accès non autorisé, en révélant la structure des identifiants internes de l'ARPT et des opérateurs.

Recommandation : Ne jamais exposer les emails de comptes réels dans le code client. Utiliser des identifiants de démonstration génériques (`demo@onit.gn`) ou implémenter un mécanisme de sélection de rôle côté serveur qui ne divulgue pas les identifiants réels.

2.4 Recherche d'operateur par correspondance floue

La fonction `getAccessibleOperators` dans `rbac.ts` utilise une recherche par correspondance partielle pour resoudre l'operateur associe a un utilisateur : la methode utilise **contains** avec uniquement le premier mot de l'organisation. Cette approche est dangereuse car un utilisateur dont l'organisation est "Orange Pirates" obtiendrait acces aux donnees de l'operateur Orange. De meme, une organisation nommee "MT" pourrait matcher avec "MTN Guinee". La resolution d'identite par correspondance de chaine est intrinsequement vulnérable aux faux positifs.

Recommandation : Utiliser une correspondance exacte (code operateur ou ID) plutot qu'une recherche partielle. Ajouter un champ `operateurId` directement sur le modele `User` pour une association deterministe et fiable.

2.5 Enregistrement d'alertes sans limitation de debit

L'endpoint **POST** `/api/alerts` permet la creation d'alertes de type `SIGNALEMENT_PUBLIC` sans authentication et sans aucune limitation de debit. Un attaquant pourrait automatiser la creation de milliers d'alertes fictives, saturant le systeme de notification et noyant les alertes legitimes dans le bruit. Le handler **PATCH** de resolution d'alertes verifie bien les permissions, mais le **POST** de creation ne limite pas le volume de soumissions.

Recommandation : Implementer une limitation de debit specifique pour les soumissions publiques d'alertes (ex: 5 soumissions par heure par adresse IP). Utiliser un middleware de rate limiting persistant (Redis) plutot que le systeme actuel en memoire.

3. Vulnerabilites de Securite Elevees

3.1 Limitation de debit en memoire non persistante

Le middleware implemente un systeme de limitation de debit base sur un Map en memoire. Ce systeme presente deux defauts majeurs : premierement, tout l'etat est perdu lors du redemarrage du serveur, permettant effectivement a un attaquant de reinitialiser ses compteurs en provoquant un redemarrage. Deuxiemement, ce mecanisme est totalement inefficace dans un deploiement multi-instances (plusieurs conteneurs ou pods), chaque instance maintenant ses propres compteurs independants. Un attaquant pourrait simplement envoyer ses requetes vers differentes instances pour contourner la limitation.

Recommandation : Migrer vers un systeme de rate limiting persistant utilisant Redis ou une base de donnees. Implementer des clés de limitation au niveau utilisateur authentifie en plus de l'IP. Ajouter des limitations specifiques par endpoint (login, creation d'alertes, import en masse).

3.2 Base de donnees SQLite en production

La configuration Docker (`docker-compose.yml`) utilise SQLite comme base de donnees de production avec `DATABASE_URL=file:/app/db/onit-png.db`. SQLite est un excellent choix pour le developpement et les applications mono-utilisateur, mais il est fondamentalement inadapté a un systeme de supervision nationale multi-utilisateurs. SQLite ne gere pas les acces concurrents en ecriture (verrouillage au niveau

fichier), ne supporte pas les connexions reseau, ne dispose pas de mecanismes de replication, et ne permet pas de scaling horizontal. Pour un observatoire national avec potentiellement des dizaines d'utilisateurs simultanes et des imports en masse de mesures QoS, cette limitation est critique.

Recommandation : Migrer vers PostgreSQL pour la production. Le schema Prisma est deja compatible avec PostgreSQL (seule la datasource du schema et l'URL changent). Configurer les variables d'environnement Docker pour pointer vers une instance PostgreSQL.

3.3 Absence de strategie de migration de base de donnees

Le Dockerfile utilise **prisma db push --skip-generate** au demarrage du conteneur. Cette commande synchronise le schema Prisma avec la base de donnees sans creer de fichiers de migration, ce qui est acceptable en developpement mais dangereux en production. En cas de modification du schema, cette commande peut silently alterer ou supprimer des colonnes de donnees existantes sans mecanisme de retour en arriere. Il n'existe aucun historique des modifications de schema, aucune possibilite de rollback, et aucune verification de coherence avant application.

Recommandation : Utiliser **prisma migrate deploy** en production au lieu de db push. Creer des migrations explicites avec **prisma migrate dev** en developpement. Versionner les fichiers de migration dans le depot de code.

3.4 Credentials hardcodes dans les fichiers de configuration

Le fichier .env.local contient des credentials en clair : le mot de passe de la base de donnees (password123), le secret NextAuth (dev-secret-not-for-production), et des cle AWS d'exemple. Si ce fichier est committe dans le depot Git, ces secrets sont accessibles a toute personne ayant acces au code source. Le docker-compose.yml exige NEXTAUTH_SECRET via une variable d'environnement, mais la configuration de developpement utilise un secret faible et previsible.

Recommandation : Verifier que .env.local est dans .gitignore. Utiliser un gestionnaire de secrets (Vault, AWS Secrets Manager) pour la production. Generer des secrets forts avec `openssl rand -base64 32`. Ne jamais committe de fichiers contenant des credentials reels.

3.5 Absence de pagination sur les endpoints de liste

L'endpoint **GET /api/users** retourne la liste complete de tous les utilisateurs sans pagination. Pour une plateforme nationale, cette liste pourrait contenir des centaines d'utilisateurs, et chaque requete charge l'integralite en memoire. Les endpoints `/api/alerts` et `/api/campaigns` presentent le meme probleme. L'endpoint `/api/mesures` implemente correctement la pagination (limit/offset), ce qui demontre que le pattern est connu mais n'a pas ete applique uniformement.

Recommandation : Implementer la pagination (limit/offset) sur tous les endpoints de liste. Appliquer une limite maximale par default (ex: 100 elements). Ajouter des liens de pagination dans les reponses (hasMore, total, nextOffset).

3.6 Absence de validation CSRF explicite

Bien que NextAuth fournisse une protection CSRF de base via des tokens dans les cookies, l'application n'ajoute aucune protection CSRF supplémentaire pour les routes API personnalisées. Les endpoints POST/PATCH/PUT acceptent des requêtes JSON sans vérification de l'origine. Un attaquant pourrait potentiellement soumettre des formulaires malveillants depuis un site tiers si l'utilisateur est déjà authentifié.

Recommandation : Vérifier l'en-tête Origin/Referer sur toutes les requêtes mutantes. Implémenter un token CSRF personnalisé pour les endpoints sensibles. Utiliser l'en-tête SameSite=Strict sur les cookies de session.

4. Problemes d'Architecture

4.1 Monolithisme frontend et absence de routing serveur

L'application est construite comme un SPA (Single Page Application) avec un routage entièrement côté client. Le fichier `page.tsx` gère tous les onglets de navigation via un état React, chargeant dynamiquement les composants dashboard sans URL distinctes. Cette architecture pose plusieurs problèmes : impossibilité de partager un lien direct vers un onglet spécifique, absence de navigation par URL (aucun historique de navigation), SEO inexistant, et difficulté à implémenter des garde-fous de sécurité au niveau du routeur.

Recommandation : Migrer vers le routage App Router de Next.js avec des pages distinctes par section (`/dashboard`, `/qos`, `/scoring`, etc.). Utiliser les layouts pour le sidebar commun. Implémenter la protection des routes au niveau du middleware Next.js.

4.2 Absence d'infrastructure de test

Le fichier `package.json` ne contient aucune dépendance de test : pas de Jest, Vitest, React Testing Library, Playwright Test, ni Cypress. Il n'existe aucun script de test configuré. Pour une plateforme gouvernementale critique de supervision des télécommunications, cette absence est inacceptable. Les régressions ne peuvent être détectées que manuellement, et la confiance dans les modifications du code est nulle.

Recommandation : Ajouter Vitest pour les tests unitaires, React Testing Library pour les composants, et Playwright pour les tests end-to-end. Cibler un minimum de 80% de couverture sur les routes API et les fonctions de sécurité. Intégrer les tests dans le pipeline CI/CD.

4.3 Duplication massive du code de mapping

Les mappings de labels de rôles (`roleLabels`), de couleurs de badges (`roleBadgeColors`), et de couleurs d'opérateurs sont dupliqués dans au moins 5 composants différents : `onit-layout.tsx`, `login-modal.tsx`, `dashboard-admin.tsx`, `user-menu.tsx`, et les fichiers d'API. Toute modification d'un rôle ou d'une couleur nécessite des changements coordonnés dans tous ces fichiers, avec un risque élevé d'incohérence.

Recommandation : Centraliser les constantes de mapping dans des fichiers dédiés (`lib/constants/roles.ts`, `lib/constants/operators.ts`). Exporter et importer ces constantes partout où nécessaire. Utiliser des variables CSS pour les couleurs côté client.

4.4 Casts de type repetes et non securises

Le pattern (`session.user as Record<string, unknown>`).`role` est repete plus de 30 fois a travers le codebase, dans chaque route API et chaque composant frontend. Ce pattern est fragile car il suppose une structure de session specifique sans verification formelle. Si la structure de la session change, aucune erreur de compilation ne sera generee, et les erreurs ne se manifesteront qu'au runtime.

Recommandation : Creer des utilitaires de type securise (ex: `getUserRole(session)`, `getUserOrg(session)`) qui centralisent les casts et ajoutent des verifications. Definir une interface `NextAuth Session` etendue correctement typee.

4.5 Absence de health check fonctionnel

L'endpoint `/api/route.ts` retourne simplement `{ message: "Hello, world!" }` sans verifier l'etat reel du systeme. Le Dockerfile configure un healthcheck sur `/api/auth/session`, ce qui est meilleur mais ne verifie pas la connectivite base de donnees ni l'etat des services externes. Un veritable health check devrait verifier la connexion a la base de donnees, la disponibilite des services externes, et l'etat general de l'application.

Recommandation : Creer un endpoint `/api/health` qui verifie : connexion DB (`SELECT 1`), disponibilite du stockage S3, etat du systeme d'authentification, et version de l'application. Utiliser cet endpoint pour le Docker HEALTHCHECK.

4.6 Absence de gestion des transactions

Les operations multi-etapes comme la creation d'utilisateur avec hashage du mot de passe, ou la resolution d'alertes avec journalisation d'audit, ne sont pas encapsulees dans des transactions Prisma. Si l'etape de journalisation echoue apres une modification reussie, le systeme se retrouve dans un etat incoherent sans trace de l'operation effectuee. Pour un systeme d'audit reglementaire, cette coherence est essentielle.

Recommandation : Utiliser `db.$transaction()` pour les operations multi-etapes critiques. Implementer des mecanismes de compensation pour les operations qui ne peuvent pas etre transactionnelles (ex: envoi d'email).

4.7 Schema Prisma : index manquants

Le schema Prisma manque d'index sur les champs frequemment requetes. Les champs `operateurId`, `regionId`, `campagneId` de `MesureQoS` sont utilises dans presque toutes les requetes de filtrage mais n'ont pas d'index dedie. De meme, le champ `statut` de `Campagne` et le champ `isResolved` de `Alerte` sont filtres regulierement sans index. Avec la croissance des donnees (potentiellement des millions de mesures QoS), les performances se degradent significativement.

Recommandation : Ajouter des index composites sur les colonnes de filtrage frequent. Exemples : `@@index([operateurId, regionId])` sur `MesureQoS`, `@@index([statut])` sur `Campagne`, `@@index([isResolved, createdAt])` sur `Alerte`.

4.8 Absence de logging structure externe

Le systeme de logging actuel se limite a console.error avec des messages en texte brut. Il n'y a pas de niveaux de log configurables, pas de rotation des logs, pas d'envoi vers un systeme d'agregation centralise (ELK, Datadog, etc.), et pas de distinction entre les environnements. Pour un systeme de supervision national, la capacite d'analyser les incidents retroactivement est cruciale.

Recommandation : Implementer un systeme de logging structure avec niveaux (debug, info, warn, error). Configurer l'envoi des logs vers un systeme d'agregation. Ajouter des correlation IDs pour tracer les requetes a travers les differents services.

4.9 Compatibilite des dependances

Le projet utilise Next.js 16 avec next-auth version 4, qui est concu pour Next.js 13-14. Le paquet @auth/prisma-adaptor est installe mais est concu pour Auth.js v5, creant une incompatibilite potentielle. Zod v4 est utilise, mais l'API de Zod a evolue significativement entre v3 et v4, et certaines methodes utilisees pourraient ne pas fonctionner correctement. L'utilisation conjointe de Prisma 6 et du client Prisma 5 pourrait egalement poser des problemes de compatibilite.

Recommandation : Aligner les versions des dependances. Migrer vers next-auth v5 (Auth.js) ou rester sur une version de Next.js compatible avec v4. Verifier la compatibilite de Zod v4 avec les schemas definis. Utiliser une version coherente de Prisma.

5. Qualite de Code

5.1 Points positifs identifies

Malgre les vulnerabilites identifiees, le codebase presente plusieurs pratiques louables qui meritent d'etre soulignees et preservees lors des corrections :

- **Validation Zod systematique :** Toutes les routes POST/PUT/PATCH utilisent des schemas Zod pour valider les entrees, avec des transformations de nettoyage HTML (stripHtml) preventives contre les XSS.
- **Journalisation d'audit :** La plupart des operations sensibles sont enregistrees dans la table AuditLog avec l'identifiant utilisateur, l'action, la ressource et les details.
- **Nettoyage HTML :** La fonction stripHtml est appliquee systematiquement sur les champs texte des schemas Zod, prevenant les injections de scripts dans les donnees persistees.
- **En-tetes de securite :** Le middleware ajoute des en-tetes de protection (X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, Referrer-Policy, Permissions-Policy) a toutes les reponses.
- **Cookies securises :** La configuration NextAuth adapte les cookies securises selon l'environnement (HTTPS en production, HTTP en developpement) avec les flags httpOnly et sameSite.
- **RBAC structure :** Le systeme de controle d'accès base sur les roles avec des politiques d'accès aux donnees (RLS) est bien concu et granulaire.
- **Non-exposition du mot de passe :** Le schema Prisma utilise passwordHash et les reponses API n'exposent jamais le hash du mot de passe.
- **Prevention de desactivation self :** Un administrateur ne peut pas desactiver son propre compte.

5.2 Problemes de duplication de code

La duplication de code est repandue a travers le codebase. Outre les mappings de roles et couleurs deja mentionnes, on retrouve des patterns repetes non extraits en fonctions partagees : la resolution d'ID operateur a partir d'un code est implementee independamment dans /api/mesures, /api/alerts, /api/scores, et /api/qos. La logique de calcul de couverture (filtrage par rssi > -100, calcul de pourcentage) est dupquee entre /api/dashboard et /api/map. La fonction formatTimeAgo est implementee trois fois separement. La logique de construction des filtres RLS (spread operateur avec conditions scope) est reperee dans chaque route API sans abstraction.

Recommandation : Extraire les logiques partagees dans des fonctions utilitaires : resolveOperatorId(code), calculateCoverage(measures), formatTimeAgo(date), buildRLSFilter(scope, opIds, regIds). Creer un hook React useApiWithRLS pour standardiser les appels API cote client.

5.3 Gestion des erreurs insuffisante

La gestion des erreurs est uniforme et basique : chaque route API catch les erreurs et retourne un generic "Erreur serveur" avec un status 500. Il n'y a pas de distinction entre les erreurs de validation (400), les erreurs d'autorisation (403), les ressources introuvables (404), et les erreurs serveur internes (500). Le logging des erreurs se limite a console.error sans contexte suffisant pour le diagnostic. Les erreurs Prisma specifiques (contrainte unique, enregistrement introuvable) ne sont pas traduites en reponses HTTP appropriees.

Recommandation : Implementer des classes d'erreur personnalisees (NotFoundError, AuthorizationError, ValidationError) avec des handlers centralises. Ajouter des correlation IDs dans les reponses d'erreur pour faciliter le diagnostic. Logger les erreurs avec suffisamment de contexte (endpoint, parametres, userId) sans exposer d'informations sensibles.

5.4 Absence de documentation technique

Le projet ne contient aucune documentation technique : pas de README detaille, pas de documentation d'API, pas de guide de deploiement, pas de documentation d'architecture. Les seuls commentaires present dans le code sont des separateurs decoratifs. Pour une plateforme gouvernementale qui sera potentiellement maintenue par differentes equipes au fil du temps, cette absence rend la maintenance extremement difficile et couteuse.

Recommandation : Creer un README.md complet avec les instructions d'installation, de configuration et de deploiement. Documenter l'architecture RBAC et les politiques RLS. Generer une documentation d'API avec Swagger/OpenAPI. Documenter les procedures de sauvegarde et restauration.

5.5 Problèmes de configuration Docker

La configuration Docker presente plusieurs problemes de securite et de bonnes pratiques. Le Dockerfile copie les scripts de seed dans l'image de production, ce qui est inutile et augmente la surface d'attaque. L'instruction COPY --from=builder /app/db ./db copie potentiellement la base de donnees de developpement dans l'image. Le script de demarrage execute prisma db push a chaque demarrage, ce qui est dangereux comme mentionne precedemment. Les limites de ressources (memoire, CPU) ne sont pas

configurees dans docker-compose.yml.

Recommandation : Nettoyer le Dockerfile pour ne copier que les fichiers necessaires en production. Utiliser prisma migrate deploy au lieu de db push. Configurer les limites de ressources. Ajouter un utilisateur non-root pour l'execution (deja partiellement fait avec nextjs).

6. Recommendations Prioritaires

Les recommandations suivantes sont classees par priorite et estimees en effort d'implementation. Les corrections critiques doivent etre appliquees avant tout deploiement en production.

Priorite	Recommandation	Effort
CRITIQUE	Ajouter authentication sur tous les endpoints sensibles	2 jours
CRITIQUE	Supprimer les falsifications de donnees (zonesBlanches + 200, Math.random())	1 jour
CRITIQUE	Retirer les emails de comptes du code client	0.5 jour
CRITIQUE	Corriger la resolution d'operateur (correspondance exacte)	1 jour
HAUTE	Migrer vers PostgreSQL pour la production	3 jours
HAUTE	Implementer le rate limiting persistant (Redis)	2 jours
HAUTE	Ajouter la pagination sur tous les endpoints de liste	2 jours
HAUTE	Migrer vers prisma migrate deploy en production	1 jour
HAUTE	Securiser les fichiers de configuration et les secrets	1 jour
HAUTE	Ajouter la protection CSRF	1 jour
MOYENNE	Ajouter les index Prisma manquants	1 jour
MOYENNE	Implementer l'infrastructure de test (Vitest + Playwright)	5 jours
MOYENNE	Refactorer le code duplique (mappings, utilitaires)	3 jours
MOYENNE	Migrer vers le routage App Router de Next.js	5 jours
BASSE	Ajouter la documentation technique	3 jours
BASSE	Implementer le logging structure	2 jours

Tableau 3 : Plan d'action prioritaire

7. Matrice Complete des Vulnerabilites

ID	Description	Severite	Categorie	Section
V-001	Acces non authentifie /api/dashboard	CRITIQUE	Securite	2.1
V-002	Acces non authentifie /api/map	CRITIQUE	Securite	2.1
V-003	Acces non authentifie /api/scoring	CRITIQUE	Securite	2.1
V-004	Falsification donnees (zonesBlanches + 200)	CRITIQUE	Integrite	2.2
V-005	Tailles de rapports aleatoires (Math.random)	CRITIQUE	Integrite	2.2
V-006	Donnees de repli non identifiees comme estimees	CRITIQUE	Integrite	2.2
V-007	Exposition emails de comptes dans le code client	CRITIQUE	Securite	2.3
V-008	Resolution operateur par correspondance floue	CRITIQUE	Securite	2.4
V-009	Alertes publiques sans limitation de debit	HAUTE	Securite	2.5
V-010	Rate limiting en memoire non persistant	HAUTE	Securite	3.1
V-011	SQLite en production	HAUTE	Architecture	3.2
V-012	Absence de migrations (prisma db push en prod)	HAUTE	Architecture	3.3
V-013	Credentials hardcodes dans .env.local	HAUTE	Securite	3.4
V-014	Absence de pagination (users, alerts, campaigns)	HAUTE	Performance	3.5
V-015	Absence de protection CSRF explicite	HAUTE	Securite	3.6
V-016	Architecture SPA sans routing serveur	MOYENNE	Architecture	4.1

V-0 17	Absence d'infrastructure de test	MOYEN NE	Qualite	4.2
V-0 18	Duplication massive du code de mapping	MOYEN NE	Qualite	4.3
V-0 19	Casts de type repetes et non securises	MOYEN NE	Qualite	4.4
V-0 20	Health check non fonctionnel	MOYEN NE	Architecture	4.5
V-0 21	Absence de gestion des transactions	MOYEN NE	Architecture	4.6
V-0 22	Index Prisma manquants	MOYEN NE	Performance	4.7
V-0 23	Absence de logging structure externe	MOYEN NE	Observabilite	4.8
V-0 24	Incompatibilite de dependances (next-auth v4 + Next.js 16)	MOYEN NE	Architecture	4.9
V-0 25	Duplication de la logique de resolution d'ID operateur	BASSE	Qualite	5.2
V-0 26	Duplication du calcul de couverture	BASSE	Qualite	5.2
V-0 27	Duplication de formatTimeAgo	BASSE	Qualite	5.2
V-0 28	Gestion des erreurs generique et insuffisante	BASSE	Qualite	5.3
V-0 29	Absence de documentation technique	BASSE	Qualite	5.4
V-0 30	Problemes de configuration Docker	BASSE	DevOps	5.5
V-0 31	Donnees mock non separees du code de production	BASSE	Qualite	-
V-0 32	Absence de limites de ressources Docker	BASSE	DevOps	-
V-0 33	Composant cyber dashboard sans donnees reelles	BASSE	Fonctionnel	-
V-0 34	Absence de mecanisme de changement de mot de passe	BASSE	Securite	-

Tableau 4 : Matrice complete des vulnerabilites identifiees

8. Conclusion

La plateforme ONIT-PNG presente une base fonctionnelle solide avec un systeme RBAC bien concu, une validation des entrees rigoureuse via Zod, et une journalisation d'audit adequate. Cependant, les vulnerabilites de securite identifiees, notamment l'accès non authentifié a des donnees sensibles et la falsification d'indicateurs, necessitent une correction immediate avant toute mise en production. La migration vers PostgreSQL, l'ajout d'une infrastructure de test, et la refactorisation du code duplique sont des investissements necessaires pour garantir la fiabilite, la securite et la maintenabilite a long terme de l'observatoire national.

L'effort total estime pour corriger les vulnerabilites critiques et elevees est d'environ **15 jours-homme**, tandis que les ameliorations architecturales et de qualite representent environ **20 jours-homme** supplementaires. Il est recommande de prioriser les corrections de securite et de planifier les ameliorations architecturales sur les sprints suivants.